Sorting & Searching

The Backbone of efficient programming

Ajay Iyengar
Email: iyengarajay@gmail.com

Introduction to Sorting

- Sorting is the process of arranging data in a meaningful order
  - Alphabetical, by size, ascending/descending
- Real world Sorting Examples
  - Your Phone Contacts - Alphabetically sorted for quick access
  - Netflix Movies - Sorted by ratings, release date, or popularity
  - Amazon Search Results - Sorted by price, customer reviews, or relevance
  - Your Email Inbox - Sorted by date, sender, or importance
  - Spotify Playlists - Sorted by artist, genre, or your listening frequency

Why is Sorting so Important ?

- **Improves User Experience**
  - E-commerce sites sort products by price, ratings, popularity
  - Social media feeds sort posts by latest or relevance
  - Maps applications sort routes by travel time
- **Makes Searching Lightning Fast**
  - Finding a contact in your sorted phone book: seconds
  - Finding the same contact in an unsorted list: minutes (or hours!)
- **Enables Advanced Algorithms**
  - Many powerful algorithms REQUIRE sorted data to work
  - Database queries run exponentially faster on sorted data

Real Performance impact of Sorting

Imaging finding an item among 1 million records -

- Unsorted data: = up to 1 million checks
- Sorted data: Finding the same item = maximum 20 checks!

That's the difference between waiting 10 seconds vs 0.001 seconds!

Why learn different Sorting Algorithms ?

- If sorting is so important, why not just use one 'best' algorithm?
- The Reality: **No Single Algorithm Rules Them All**
- Different situations need different approaches
  - **Bicycle** 🚲 : Great for short distances (small data)
  - **Car** 🚗 : Perfect for medium distances (moderate data)
  - **Airplane** ✈️ : Essential for long distances (massive data)
  - **Boat** ⛵ : Best when you need to carry heavy cargo (memory-efficient)

Different situations = Different approaches

- **Very small datasets** (≤ 10-20 items): Simple algorithms like ***Insertion Sort*** often win due to low overhead
- **Large datasets** (thousands to millions): Advanced algorithms like ***Merge Sort*** or ***Quick Sort*** are essential
- **Nearly sorted data**: Insertion Sort can be very efficient (O(n) best case vs O(n log n) for others

Different situations = Different approaches

- **Memory constraints**: Quick Sort uses O(log n) extra space vs Merge Sort's O(n) extra space
- **Stability required**: Merge Sort preserves original order of equal elements; Quick Sort doesn't guarantee this
- **Minimal memory writes**: ***Selection Sort*** minimizes the number of swaps (useful for expensive write operations)
- **Educational purposes**: ***Bubble Sort*** and Selection Sort are excellent for understanding sorting concepts, though rarely used in production

What Does Java Actually Use?

Java 1.2 → Java 6

- **Primitives (int[], double[], etc.)** → Classic Quick Sort (single-pivot).
- **Objects (Integer[], String[], etc.)** → Merge Sort (stable).
- Good performance, but had worst-case $O(n^2)$ for primitive

Java 7+ (2011)

- **Primitives → Dual-Pivot Quick Sort** (Yaroslavskiy's algorithm).
  - Faster in practice.
  - Falls back to Insertion Sort for very small arrays (< 47).
- **Objects → TimSort** (hybrid of Merge Sort + Insertion Sort).
  - Stable.
  - Optimized for real-world data (often partially sorted).

Key takeaways

Arrays.sort() or Collections.sort()

- Primitives → Dual-Pivot Quick Sort + Insertion Sort.
- Objects → TimSort (hybrid of Merge Sort + Insertion Sort)
- Different algorithms for primitives vs objects.
- No Heap Sort fallback in Java (unlike C++'s introsort).
- TimSort dominates for objects because real-world data is rarely random — it's often *partially ordered*.

```
Arrays.sort(myIntArray);        // Uses Dual-Pivot Quicksort hybrid
Collections.sort(myList);       // Uses Timsort
```

Internal Sorting – C++ & Python

Python

- **Built-in list.sort() and sorted():**
  - **TimSort** (same as Java objects).
  - Stable.
  - Excellent for *real-world data* (partially sorted sequences).

C++

- **std::sort():**
  - **Introsort** (Quick Sort + Heap Sort fallback + Insertion Sort for small arrays).
  - ⚡ Guarantees O(n log n) worst case.
  - Not stable.
- **std::stable_sort():**
  - **Merge Sort variant** (stable).

What is Searching ?

- 👉 **Finding an item in a collection of data.**
- 💡 Real Life Examples:
  - Typing a song name on Spotify and getting the result instantly.
  - Looking up a word in a dictionary.
  - Searching a friend on Instagram.
  - Finding your transactions among thousands in a banking App

Every time you press Ctrl+F to find text on a webpage, you're using a search algorithm!

Why is Searching Crucial ?

- **Foundation of Everything You Use**
  - Google → answers billions of queries every day
  - Netflix/Amazon → find movies or products in huge catalogs
  - WhatsApp → instantly finds your contacts
  - Databases → locate the right record among millions
- **Makes Applications Interactive**
  - Auto-complete suggestions as you type in Google, YouTube
  - Real-time filtering in shopping apps (price, brand, rating)
  - Instant messaging contact lookup

‘Searching’ Reality Check

- **Searching an item in 1 billion records(1,000,000,000)**
  - **Linear Search** : Up to 1 billion operations
  - **Binary Search** in same data: Maximum 30 operations!

That's the difference between your app responding instantly vs taking hours!

Why learn Searching Algorithms ?

🔎 Because data is huge, and looking line by line takes forever!

🎯 Practical Reason: Searching powers everything digital – from databases to Google search.

Next time you get a Google result in 0.2 seconds, remember—underneath, it’s optimized searching at work!

Sorting + Searching

Sorting and Searching often go hand-in-hand

💡 Example:

- Sort your Netflix movies by genre (sorting).
- Find the exact movie you want (searching).

👉Together, they form the foundation of **efficient data handling** in computer science.

Bubble Sort

The Lazy Librarian's Sorting Adventure! 📚

- Meet Pakya, the Librarian – The laziest(yet most systematic) librarian in town
- He can only compare 2 books at a time, and only if they are sitting right next to each other
- Bubble sort is exactly like Pakya's method

What is Bubble Sort ?

- 🔍 **Compare only adjacent elements** (Pakya's "one book at a time" rule)
- 🔄 **Swap them if they're in wrong order** (Pakya moves the bigger book to the right)
- 🎯 **Repeat until everything is sorted** (Pakya keeps walking the shelf until no more swaps needed)
- Pakya’s philosophy

```
if (leftBook.size > rightBook.size) {
    swap(leftBook, rightBook); // "This big book doesn't belong here!"
}
```

Let's bubble sort

[5 , 3 , 8 , 4 ,2]

Pass 1 (4 comparisons):

- Compare (5,3) → swap → [3] [5] [8] [4] [2]
- Compare (5,8) → no swap
- Compare (8,4) → swap → [3] [5] [4] [8] [2]
- Compare (8,2) → swap → [3] [5] [4] [2] [8] ✅

👉 Largest (8) bubbled to the end

Pass 2 (3 comparisons):

- Compare (3,5) → no swap
- Compare (5,4) → swap → [3] [4] [5] [2] [8]
- Compare (5,2) → swap → [3] [4] [2] [5] [8] ✅

👉 Next largest (5) settled

Pass 3 (2 comparisons):

- Compare (3,4) → no swap
- Compare (4,2) → swap → [3] [2] [4] [5] [8] ✅

Pass 4 (1 comparison):

- Compare (3,2) → swap → [2] [3] [4] [5] [8] ✅

👉 Sorted

Pass	Comparisons
1	4
2	3
3	2
4	1

n=5

Total passes = n-1

Comparisons **for each** pass i = n-i

Bubble Sort Code : First Cut

- Will it work ?

Pass	Comparisons
1	4
2	3
3	2
4	1

```
3           int arr[]={5,3,8,4,2};
4           int n=arr.length;
5           for(int i=0;i<n-1;i++){
6               for(int j=0;j<n-i;j++){
7                   if(arr[j]>arr[j+1]){
8                       int temp=arr[j];
9                       arr[j]=arr[j+1];
10                      arr[j+1]=temp;
11                  }
12              }
13          }
```

n=5

Total passes = n-1

Comparisons **for each** pass *i* = n-i

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of b
ounds for length 5
        at BubbleSortBuggy.main(BubbleSortBuggy.java:7)
```

Bubble Sort Code : Second Cut

- Will it work ?

Pass	Comparisons
1	4
2	3
3	2
4	1

```
3       int arr[]={5,3,8,4,2};
4       int n=arr.length;
5       for(int i=0;i<n-1;i++){
6           for(int j=0;j<n-i-1;j++){
7               if(arr[j]>arr[j+1]){
8                   int temp=arr[j];
9                   arr[j]=arr[j+1];
10                  arr[j+1]=temp;
11              }
12          }
13      }
```

n=5

Total passes = n-1

Comparisons **for each** pass *i* = n-i

Notice the n-i-1 - this is done due to comparison between j and j+1 & fixes the ArrayIndexOutOfBoundsException

Bubble Sort Code : Third Cut

Give meaningful names to variables

Pass	Comparisons
1	4
2	3
3	2
4	1

noOfElements=5
Total passes = noOfElements-1
Comparisons **for each** pass *i* = noOfElements-i

```
int arr[]={5,3,8,4,2};
int noOfElements=arr.length;
for(int i=0;i<noOfElements-1;i++){
    for(int j=0;j<noOfElements-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j]; //can move to a function
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```

Refactoring code to function

Bubble Sort : Can we improve ?

[3 , 2 , 1 , 4 , 5]

3 , 2 , 1 , 4 , 5

Pass 1 (i = 0) – 4 comparisons

- Compare (3,2) → swap → [2, 3, 1, 4, 5]
- Compare (3,1) → swap → [2, 1, 3, 4, 5]
- Compare (3,4) → no swap
- Compare (4,5) → no swap

2 , 1 , 3 , 4 , 5

Pass 2 (i = 1) – 3 comparisons

- Compare (2,1) → swap → [1, 2, 3, 4, 5]
- Compare (2,3) → no swap
- Compare (3,4) → no swap

1 , 2 , 3 , 4 , 5

Pass 3 (i = 2) – 2 comparisons

- Compare (1,2) → no swap
- Compare (2,3) → no swap

Do we need a 4th Pass ?

Pass	Comparisons	Swaps Made	Status After Pass	Continue?
1	4	2	`[2 1 3 4 5]`	✅ Continue
2	3	1	`[1 2 3 4 5]`	✅ Continue
3	2	0	`[1 2 3 4 5]`	🚫 Stop

noOfElements=5
Total passes = noOfElements-1 = 4
Comparisons **for each** pass *i* = noOfElements-i

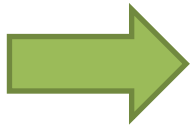
Bubble Sort : Can we improve ?
[2 , 3 , 4 , 5 , 8]

2, 3 , 4 , 5 , 8

Pass 1 (i = 0)

- Compare (2,3) → no swap
- Compare (3,4) → no swap
- Compare (4,5) → no swap
- Compare (5,8) → no swap

👉 No swaps at all → Array already sorted! ✅



- When there are no swaps at all for a particular pass – we stop
- We add a flag to check if swap happened during a pass
- If no swaps - break

Do we even need a 2nd Pass ?

noOfElements=5
Total passes = noOfElements-1 = 4
Comparisons **for each** pass *i* = noOfElements-i

Bubble Sort Code : Optimized

```java
int arr[] = {5,3,8,4,2};
int noOfElements = arr.length;
for(int i=0;i<noOfElements-1;i++){
    boolean swapped = false;
    for(int j = 0;j < noOfElements-i-1;j++){
        if(arr[j] > arr[j+1]){
            int temp = arr[j]; //can move to a function
            arr[j]=arr[j+1];
            arr[j+1]=temp;
            swapped=true;
        }
    }
    if(!swapped){
        break;
    }
}
```

Key traits of Bubble Sort

- Lazy but reliable – Sometimes being lazy is actually smart
- Stable – Won't mess up on order of identical books
- Space-Conscious – Works right on the shelf(in-place sorting)
- Simple, Intuitive, and great for learning
- Almost never used in real-world production code

Just like air bubbles rise to the surface in your soda, ***the largest elements "bubble up" to their correct positions*** *at the end of each pass!*

How fast is Bubble Sort?

- Worst Day Ever – $O(n^2)$
  - Books on shelf are completely backwards
  - Books are in reverse order [90, 64, 34, 25, 22, 12, 11]
- Average Tuesday - $O(n^2)$
  - Random messy shelf
- Best Day Ever – O(n)
  - Shelf is already perfect
  - Book are already sorted
  - One walk through = n-1 comparisons

How much space does it need ?

- **Space Complexity - O(1)**
  - **Extra Shelves Needed**: ZERO!
  - **Tools Required**: Just 3 variables which will be constant no matter the size of the input array

- Variables created
  - noOfElements,temp,swapped

```java
int arr[] = {5,3,8,4,2};
int noOfElements = arr.length;
for(int i=0;i<noOfElements-1;i++){
    boolean swapped = false;
    for(int j = 0;j < noOfElements-i-1;j++){
        if(arr[j] > arr[j+1]){
            int temp = arr[j]; //can move to a function
            arr[j]=arr[j+1];
            arr[j+1]=temp;
            swapped=true;
        }
    }
    if(!swapped){
        break;
    }
}
```

Bubble Sort : Key Takeaways

- Bubble Sort = Adjacent comparisons + swaps + repeat
- Great for learning, terrible for production
- $O(n^2)$ time, $O(1)$ space, stable algorithm
- Optimization matters: early termination saves the day!
- Visual and intuitive - perfect stepping stone to advanced sorting

Every expert was once a beginner, and every beginner starts with Pakya! 🚀

Selection Sort

Why settle for swapping neighbours when you can find the best every time

- Think **"Talent Show Auditions"**!
  - 🌟 **SELECT** the best performer from remaining contestants
  - 🏆 **PLACE** them in the winner's lineup (sorted portion)
  - 🔄 **REPEAT** with remaining contestants
- **Explain Selection Sort in 10 seconds?"**

  "Find smallest, put it first. Find next smallest, put it second. Repeat!"

Selection Sort – The philosophy

- **SCAN the entire remaining array** to find the minimum
- **SELECT the absolute smallest element**
- **SWAP it with the first unsorted position**
- **MARK that position as "done forever"**
- **REPEAT** with the remaining unsorted portion

```
int minimum = findSmallestInRemaining(array, startFrom);
swap(array, currentPosition, minimum); // "Perfect placement!"
```

Let's select and sort
[64, 25, 12, 22, 11, 90]

First Mission : Position 0

```
Array: [64, 25, 12, 22, 11, 90]
       ↑
          "I need the smallest element for position 0!"

🔍 SCANNING: "Let me check all remaining elements..."
   64 (current minimum) → 25 (new minimum!) → 12 (new minimum!)
   → 22 (nope) → 11 (NEW MINIMUM!) → 90 (nope)

🎯 SELECTED: 11 (the absolute smallest)

🔄 SWAPPING: Position 0 (64) ↔ Position 4 (11)
   Result: [11, 25, 12, 22, 64, 90]
           ✅ ←(sorted)
```

Second Mission : Position 1

```
Array: [11, 25, 12, 22, 64, 90]
          ↑
          "Position 0 is done. Now I need smallest from remaining!"

🔍 SCANNING remaining [25, 12, 22, 64, 90]:
   25 (current min) → 12 (new min!) → 22 (nope) → 64 (nope) → 90 (nope)

🎯 SELECTED: 12

🔄 SWAPPING: Position 1 (25) ↔ Position 2 (12)
   Result: [11, 12, 25, 22, 64, 90]
           ✅  ✅ ←(sorted)
```

- **Mission 3**: Find min from [25, 22, 64, 90] → 22 → `[11, 12, 22, 25, 64, 90]`
- **Mission 4**: Find min from [25, 64, 90] → 25 (no swap needed!) → `[11, 12, 22, 25, 64, 90]`
- **Mission 5**: Find min from [64, 90] → 64 (no swap needed!) → `[11, 12, 22, 25, 64, 90]`

🏁 **Final Result:** `[11, 12, 22, 25, 64, 90]`

Selection Sort : First Cut

[64, 25, 12, 22, 11, 90]

```
3           int arr[] = {64,25,12,22,11,90};
4           int size = arr.length;
5           for(int i = 0 ; i < size-1; i++){ //No of missions/passes
6               int minIndex = i; //Index of minimum element
7               for(int j = i+1;j < size;j++){
8                   if(arr[j] < arr[minIndex]){
9                       minIndex = j;
10                  }
11              }
12              int temp = arr[minIndex]; //swapping
13              arr[minIndex] = arr[i];
14              arr[i] = temp;
15          }
```

Selection Sort : Second Cut

[64, 25, 12, 22, 11, 90]

```
3       int arr[] = {64,25,12,22,11,90};
4       int size = arr.length;
5       for(int i = 0 ; i < size-1; i++){ //No of missions/passes
6           int minIndex = i; //Index of minimum element
7           for(int j = i + 1 ; j < size ; j++){
8               if(arr[j] < arr[minIndex]){
9                   minIndex = j;
10              }
11          }
12          //swap only if minIndex is not the same as 'i'
13          if(minIndex != i){
14              int temp = arr[minIndex]; //swapping
15              arr[minIndex] = arr[i];
16              arr[i] = temp;
17          }
18      }
```

"Don't swap if 'minIndex' and 'i ' are the same !

How fast is Selection Sort ?

- 🎯 The Shocking Truth - ALWAYS $O(n^2)$
- All Cases (Best, Average, Worst) = $O(n^2)$
- 💾 **Space Usage:**
  - **Space Complexity - O(1):**
  - Same as Bob - works in place!
  - Only needs variables: minIndex, temp, loop counters

Selection Sort : Key takeaways

Strengths

- **Consistent Performance**: Same speed regardless of input
- **Minimal Swaps**: Great when moving elements is expensive
- **Simple Logic**: Easy to understand and implement
- **Memory Friendly**: O(1) space usage

Weaknesses

- **Not Adaptive: Doesn't speed up on sorted data**
- **Unstable: Can change order of equal elements**
- **Still $O(n^2)$: Not suitable for large datasets**

Bubble vs. Selection

Feature	Bubble Sort	Selection Sort
Idea	Repeatedly swap adjacent elements if they are out of order. Largest element *bubbles up* in each pass.	Repeatedly find the minimum from unsorted part and place it at the correct position.
Pass Target	Largest element goes to end each pass.	Smallest element goes to beginning each pass.
Comparisons	Worst: $O(n^2)$ Best (sorted array): O(n) with early exit.	Always $O(n^2)$, even if sorted.
Best Case	Already sorted → O(n)	Already sorted → Still $O(n^2)$ comparisons, but no swaps.
Space Complexity	O(1) extra (temp + flag).	O(1) extra (temp + minIndex).

Searching…

- You’re building a feature for an e-commerce app
- You have 10 million product IDs stored in sorted order

```
[101, 205, 308, 412, 599, 678, 801, 923, 1045, 1167, ... 9,999,999]
```

- A user searches for product ID 801 – How do you find it ?

Approach 1 : Linear Search

- Searching for product ID 801 in :

```
[101, 205, 308, 412, 599, 678, 801, 923, 1045, 1167, ... 9,999,999]
```

```java
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;  // Found it!
        }
    }
    return -1;  // Not found
}
```

Analysis : Linear Search

- Time Complexity : O(n)
  - Best case: Target is at index 0 → 1 comparison
  - Worst case: Target is at the end or doesn't exist → n comparisons
  - Average case: n/2 comparisons
- **For 10 million products**: You might check up to 10 million products!

But wait... the array is **SORTED**! We're not using this crucial information!

Searching in a physical dictionary

- Think about how you search in a physical dictionary:
  - You don't start from 'A' and flip through every page
  - You open somewhere in the middle
  - If your word comes before that page, you go left
  - If it comes after, you go right
  - You keep doing this until you find it

This is Binary Search!

Binary Search – The Core Algorithm

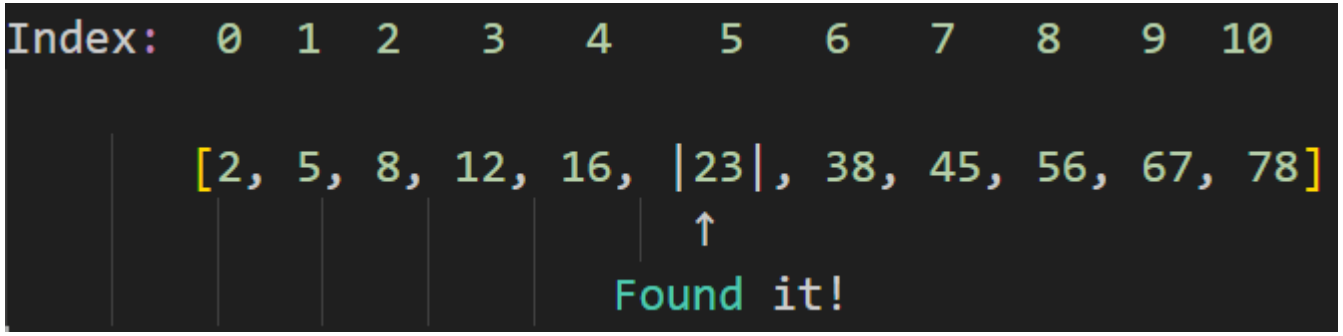
- Eliminate half of the search space in every step

- Instead of checking every element, we:
  - Check the middle element If it's our target → Done!
  - If target is smaller → Search in the left half
  - If target is larger → Search in the right half
  - Repeat until found or search space becomes empty

Binary Search – Visual Example

- Let's search for 23 in this array :

- Step 1 : Check middle (index 5)



Binary Search – Visual Example

Let's search for 67 in this array

Step 1 : Check middle (index 5)

```
Index:  0  1  2   3   4    5   6   7   8   9  10
       [2, 5, 8, 12, 16, |23|, 38, 45, 56, 67, 78]
        ←----- left ----  mid  ----- right ----→

67 > 23, so search in RIGHT half
```

Step 2 : Check middle of right half (index 8)

```
[38, 45, |56|, 67, 78]
          mid
67 > 56, so search in RIGHT half
```

Step 3 : Check middle of new right half (index 9)

```
[|67|, 78]
 mid

67 == 67, Found it!
```

Instead of 9 comparisons (linear search), we did it in 3!

Binary Search Code

```java
public static int binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        // Calculate mid (why this formula ?)
        int mid = left + (right - left) / 2;

        // Check if target is at mid
        if (arr[mid] == target) {
            return mid;  // Found it!
        }

        // If target is smaller, ignore right half
        if (arr[mid] > target) {
            right = mid - 1;
        }
        // If target is larger, ignore left half
        else {
            left = mid + 1;
        }
    }

    // Target not found
    return -1;
}
```

```java
int[] arr = {2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78};
int target = 67;
```

Iteration	left	right	mid	arr[mid]	Action
1	0	10	5	23	67 > 23, left = 6
2	6	10	8	56	67 > 56, left = 9
3	9	10	9	67	Found! Return 9

Binary Search : Tricky parts

- Why do you see mid = left + (right – left) / 2 and not (left+right) / 2 ?
- Potentially dangerous : (left+right)/2
- Imagine an array with 1.5 billion elements (very very……rare)
  - Left = 1,000,000,000 (1 billion)
  - Right = 1,400,000,000 (1.4 billion)
  - Left + Right = 2, 400 , 000, 000
  - Integer overflow (max is approx 2.1 billion)

Bug existed in Java's own Arrays.binarySearch() for 9 years before someone discovered it!

Binary Search Code : Questions

- Why left <= right and not < ? [Think of a single element]
- Why right = mid – 1 and not right = mid ? | [1,2] and target = 0

Binary Search : Time Complexity

- **Linear Search**: Check 1, 2, 3, 4, ... until found
  - Time: O(n)
- **Binary Search**: Divide the array in half each time
  - Time: O(log n)
- What does O(log n) mean?
  - Every operation, we cut the problem size in half
  - n elements → n/2 → n/4 → n/8 → ... → 1

Binary Search is crazy fast

The Power of Logarithms

Array Size (n)	Linear Search (Worst)	Binary Search (Worst)
10	10	4
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

For 1 billion elements, binary search needs only 30 comparisons maximum!

Variations of Binary Search

Variation 1: Find First Occurrence

Problem: In a sorted array with duplicates, find the **first** occurrence of target.

```
Array: [1, 2, 2, 2, 2, 3, 4, 5]
Target: 2
Answer: Index 1 (first occurrence)
```

The Trick: Even when we find the target, keep searching in the left half!

Binary Search : Find first occurrence

```java
public static int findFirstOccurrence(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;
    int result = -1;  // Store the answer

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;        // Found it, but...
            right = mid - 1;     // Keep looking LEFT for earlier occurrence
        }
        else if (arr[mid] > target) {
            right = mid - 1;
        }
        else {
            left = mid + 1;
        }
    }

    return result;
}
```

Variations of Binary Search

Variation 2: Find Last Occurrence

Problem: In a sorted array with duplicates, Find the **last** occurrence of target

```
Array: [1, 2, 2, 2, 2, 3, 4, 5]
Target: 2
Answer: Index 4 (last occurrence)
```

Binary Search : Find last occurrence

```java
public static int findLastOccurrence(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;        // Found it, but...
            left = mid + 1;      // Keep looking RIGHT for later occurrence
        }
        else if (arr[mid] > target) {
            right = mid - 1;
        }
        else {
            left = mid + 1;
        }
    }

    return result;
}
```

Binary Search in the Real World

- Database Indexing
  - When we run a "SELECT * FROM users WHERE user_id = 12345" and if user_id is indexed, the database uses a B-Tree(a fancy version of binary search)
  - Finds record in O(log n) time instead of scanning millions of rows
  - Without indexing - Linear search (slow)
- Git Bisect(Finding a bug) uses Binary search
- Network routers use algorithms based on Binary Search
- Used in many 3D games/applications & Game AI decision-making

Binary Search : Key Takeaways

- **The Binary Search Mantra** 
  - **Prerequisite**: Array MUST be sorted
  - **Idea**: Eliminate half the search space each step
  - **Time**: O(log n) - insanely fast!
  - **Space**: O(1) - very memory efficient

Binary Search : Assignments

- Standard binary search
- Given a sorted array with duplicates, find first occurrence of a target
- Given a sorted array with duplicates, find last occurrence of a target
- Given a sorted array with duplicates, count total occurrences of a target